

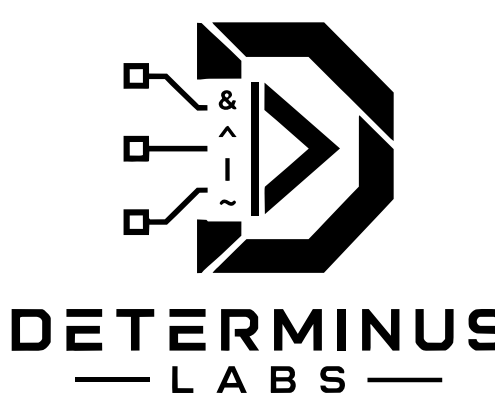
CPPCON 2026 POSTER

SAME BITS, ALL THE MIPS



More information

Andrew Drakeford
Determinus Labs Ltd
Andrew@DeterminusLabs.com
WG21 · P4016R0 canonical reduction
P4229R0 canonical scan



DETERMINUS
LABS

1

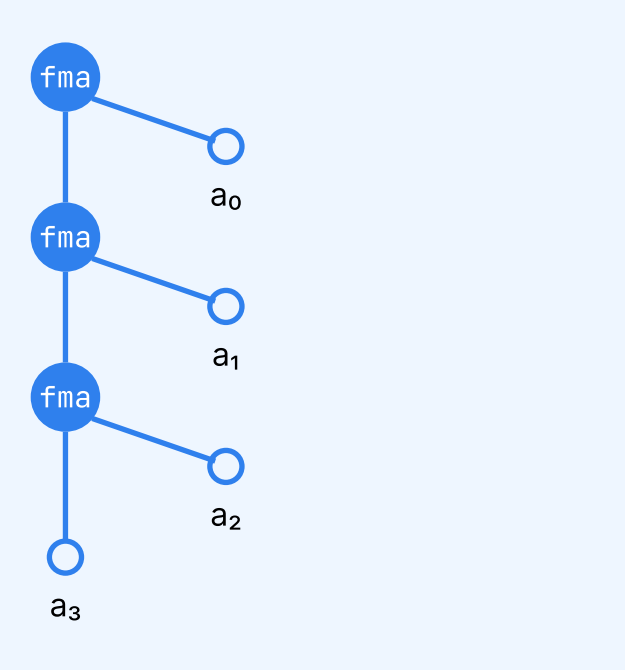
EXPRESSION GEOMETRY

Same polynomial. Different expression. Different consequences.

$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ — one mathematical object

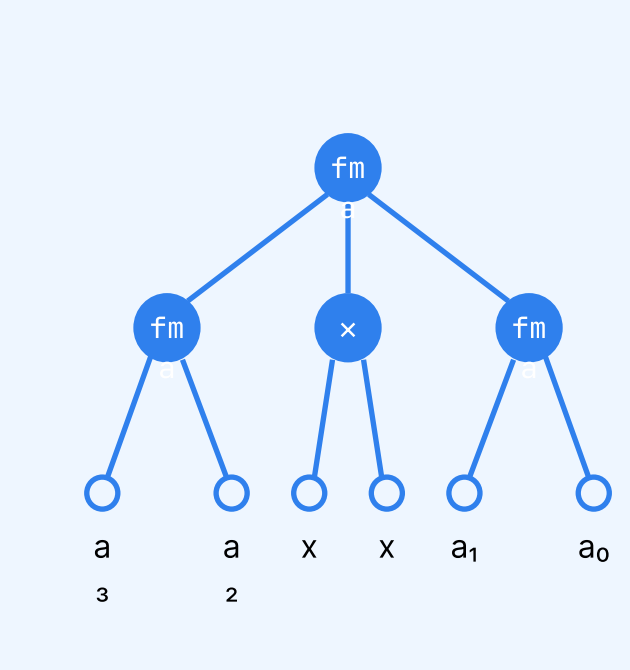
Horner

$((a_3x + a_2)x + a_1)x + a_0$



Estrin

$(a_3x + a_2)x^2 + (a_1x + a_0)$



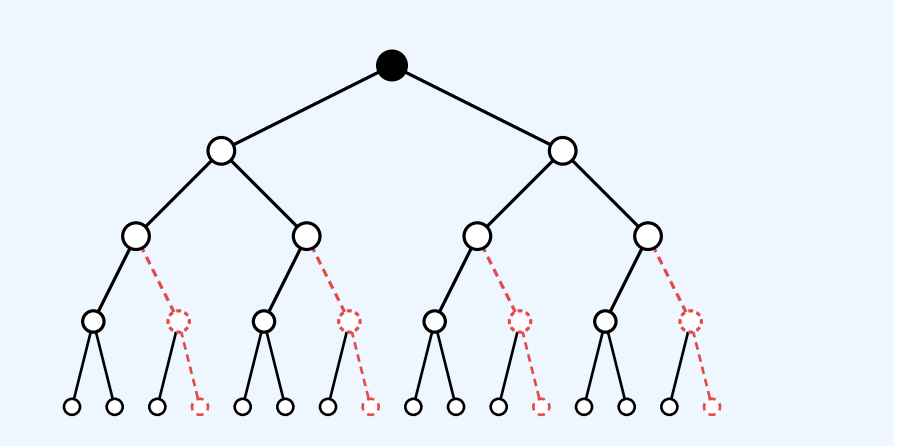
depth 3 · serial
each node: one fma, one rounding — fusing is an expression choice [reduce]: a generalized sum, grouping unspecified — **today, the tree is a hidden variable**

depth 2 · parallel

Comparing aggregation geometries

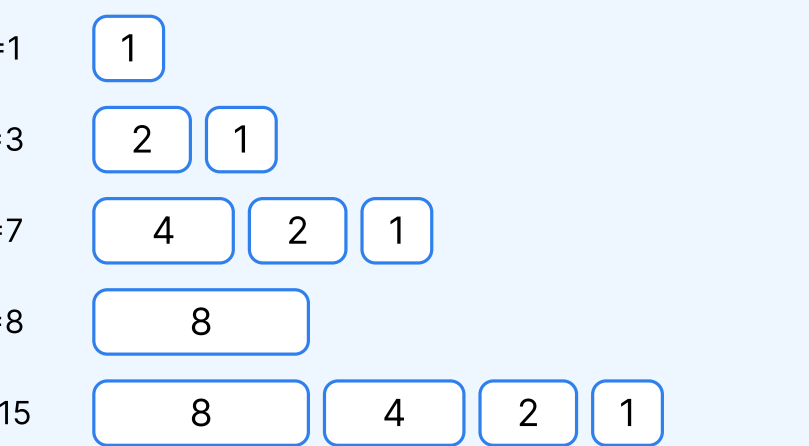
different tree families, different numerical and machine properties

Perfectly balanced binary tree



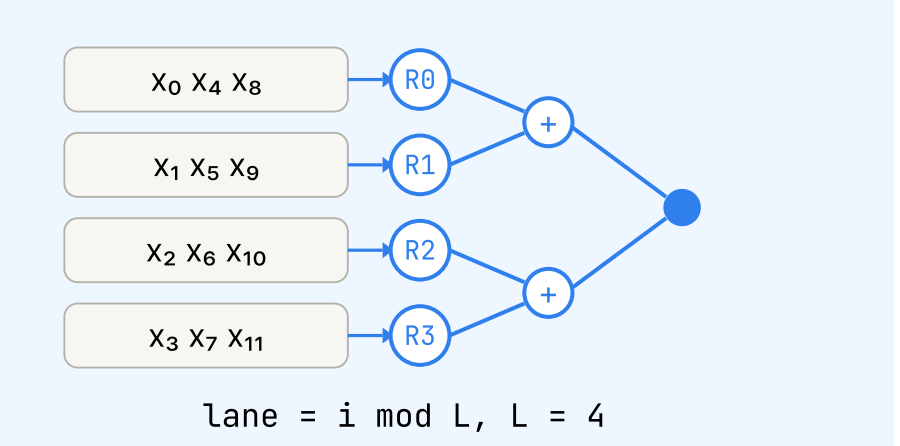
$N = 12 \rightarrow 6 + 6 \rightarrow 3 + 3 + 3 + 3$ geometry chosen after N is known; every fourth leaf slot is missing, so the whole bottom row alternates add / pass

Shift-Reduce frontier



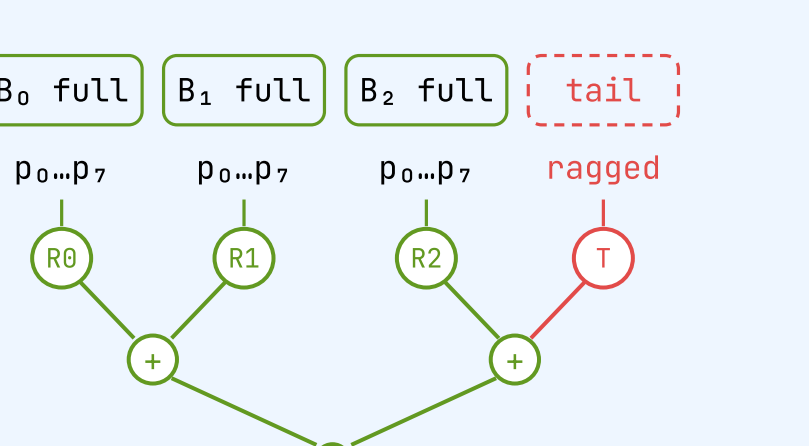
append-only: a completed 2^k span collapses to one root; one live root per occupied power of two
prefix-stable on append: yes

P4016, Interleaved, Shift-Reduce



fixed logical lanes; lane count is semantic, not the ISA width
prefix-stable on append: yes

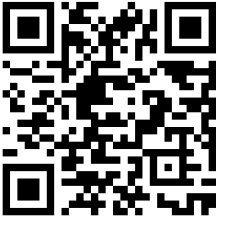
P4229, blocked dyadic reduce + scan



regular full blocks; dense local prefixes; irregularity confined to the tail
prefix-stable at block boundaries: yes

	std::accumulate	std::reduce	balanced	Shift-Reduce	P4016	P4229
log depth		✓	✓	✓	✓	✓
unique/deterministic expression	✓		✓	✓	✓	✓
prefix-stable on append				✓	✓	✓
bottom-level regularity				✓	✓	✓
streamable				✓	✓	✓
error $O(\log N)$		✓	✓	✓	✓	✓

References



Shift Reduce
Dalton, Wang, Blainey (IBM Toronto). SIMDizing pairwise sums: a summation algorithm balancing accuracy with throughput. WPMVP '14 (with PPOPP), ACM, 2014.
doi.org/10.1145/2568058.2568070



Blocked Prefix Sums
Fraser, Xu, Leiserson (MIT CSAIL). Work-efficient parallel algorithms for accurate floating-point prefix sums. IEEE HPEC 2020.
ieeexplore.ieee.org/document/9286240



Canonical Reduce
Drakeford, A. P4016R0: Canonical parallel reduction. WG21, pre-Croydon mailing, February 2026.
wg21.link/P4016r0



Canonical Scan
Drakeford, A. P4229R0: Canonical parallel scan. WG21, 2026 — companion to P4016R0.
wg21.link/P4229r0

2

COMPOSING A COMPUTATION

Softmax: contracts compose

One algorithm. Several numerical contracts. The composition used in reproducible llama2.c.

$m = \max_i z_i \quad e_i = \exp(z_i - m) \quad s = \sum_i e_i \quad y_i = e_i / s$

1 · maximum
 $m = \max_i z_i$
aggregation expression + observation

```
//member of operator struct
float left_biased_max_f32::operator()(float left, float right) const { return left < right ? right : left; }

auto maximum = canonical_block_dyadic_reduce<64>(
    left_biased_max_f32{}, input.data(),
    input.size(), -::infinity());
```

• named expression: canonical_block_dyadic<64>
• observation: reduce root
• lambda is left-biased max

2 · shifted exp
 $e_i = \exp(z_i - m)$
contracted elementwise result

```
kernel_transform(input_view,
    output_view, detail::shifted_exp_f32{maximum});

float operator()(float x){ return cm_exp(x - maximum); }
```

• block/scalar transform callable
• contracted FP32 cm_exp
• same operation over block<float, W>

3 · retain e_i
exponentials[] become the next input

```
const std::span<const float>
    exponentials{output.data(), output.size()};
```

• controlled materialization
• not claimed as one-pass softmax

4 · normalization sum
 $s = \sum_i e_i$
aggregation expression + observation

```
inline float
    softmax_sum_exp_f32_unchecked(std::span<const float> exponentials) noexcept
{
    return canonical_block_dyadic_reduce<64>(
        std::plus<float>{}, exponentials.data(),
        exponentials.size(), 0.0f);
}
```

• named expression: canonical_block_dyadic<64>
• observation: reduce root
• geometry is specified, not inferred

5 · normalize
 $y_i = e_i / s$
IEEE primitive operation

```
kernel_transform(exponentials_view, output_view,
    detail::divide_by_sum_f32{sum});
```

• block/scalar transform callable
• IEEE division at selected precision

3

COMPOSITION

Composing deterministic parallel algorithms

Building deterministic algorithms from primitives and composers

1 · Leaf helpers

zip, predicate, project, select, adjacent, key / index

2 · Core primitives

transform, gather, scatter, reduce, scan

3 · Derived blocks

flags, ranks, histogram, segmentation, prefix allocation, pack / compact, routing stable

4 · Composite algorithms

pack / copy_if, stable partition, radix sort, run-length encode, reduce-by-key, group-by, radix select / top-k, segmented pack, sparse assembly, CSR transpose, CSR SpMV, BFS frontier, softmax, segmented softmax, and more ...

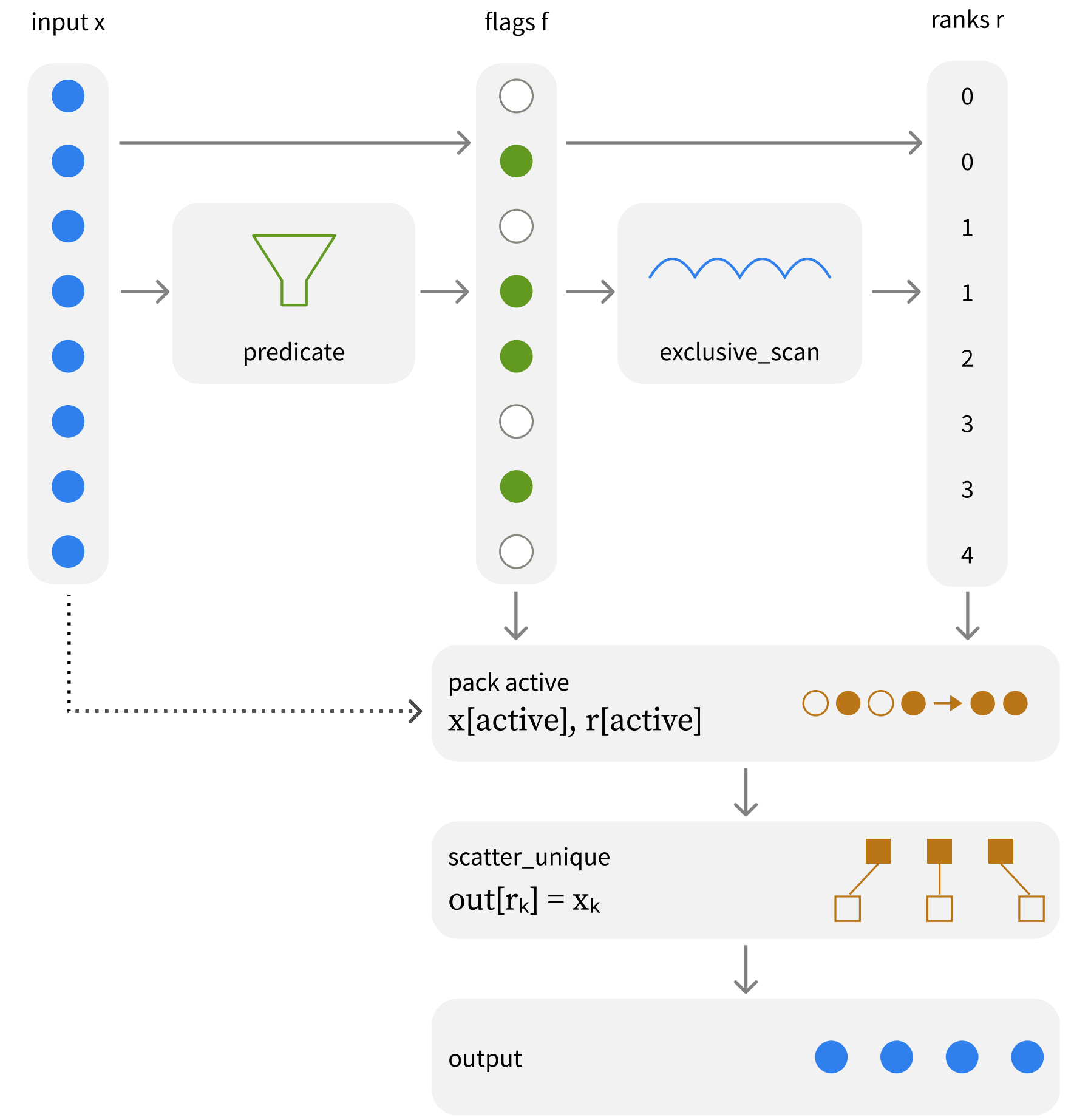
Same expression for primitives brings consistency to composites

```
scan[i] = reduce(x[0..i])
```

The last exclusive prefix plus the last flag equals reduce(+, f): scan and reduce agree on the output size, bit for bit.

copy_if

Built from leaf helpers and core primitives



```
copy_if(x, p) = scatter_unique( x[active], r[active] )

f = [ p(x) ] · r = exclusive_scan(+, f, 0) · active = { i : f_i = 1 }
```

4

APPLICATIONS

Deterministic numerics: agree the contract, keep the implementation free.

Transcendental functions
Same expression, same IEEE 754 arithmetic, same bits — on every platform and ISA.

one named expression
IEEE 754 · RN-even
no fast-math
no reassociation

x86 · AVX2
x86 · AVX-512
ARM · NEON
CUDA

same bits

Accuracy is a second dial
Turn it and the bits still agree: a lower tier is a different named expression.

correctly rounded faithful (≤ 1 ulp) wide (≤ 8 ulp)

accuracy throughput

exp throughput, FP32 — ns per element, AVX-512 W = 16, lower is better

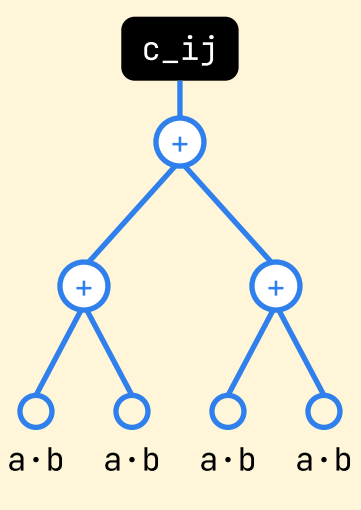


FP8 lookup 0.19 · FP4 lookup 0.15 — table lookups. FP32 wide: ≤ 1 ulp over all 2^{32} inputs.
exp · log · sin · cos · tanh · gelu ... FP64 → FP4 · CR / faithful / wide · scalar → AVX-512 / NEON / CUDA

Linear algebra
A dot product is a reduction. GEMM is that, composed.

$c_{ij} = \sum_k a_{ik} b_{kj}$

one dot product per cell;
the tree names the reduction



the panel-1 tree, at every output cell

Ascending-k FMA chain

90% of cuBLAS FP32 RTX 4090, locked clocks
rdyadic = chain $\pm 0.3\%$ at equal execution geometry

Recursive dyadic (rdyadic)

94% of OpenBLAS, CPU indicative
67% of cuBLAS RTX 4090

When the hardware will not take an order: change the contract to integers
FP4 / FP64 in → split to integers → exact INT8 MMA → one rounding → FP4 / FP64 out
Tensor cores and VNNI do not fix floating-point order. Integer sums are exact in any order.

6.0× FP64 exact vs cuBLAS DGEMM native FP64 is 1/64-rate on this card

0.83 TOPS FP4 exact, VNNI · indicative Zen 4

132 TOPS FP4 exact, cuBLASlt engine RTX 4090

Heterogeneous, bitwise-reproducible LLM inference
llama2.c running on x86 · ARM · CUDA — same bits. CPU at native speed

matmul → fixed FMA chain, dyadic combine
rmsnorm → block-dyadic reduce + rsqrt
softmax → the panel-2 composition
top-p sampler → stable sort + canonical scan (P4229)

9 numerical operations → 9 named expressions
Linux · GCC · Windows · MinGW, MSVC · macOS · M4, NEON = scalar · CUDA sm_89 (not yet timed)
stories15M · 64 steps · 12,864 checkpoints · logits c50b9583a5fa7dc1 · reproduce it: QR, top right

96.9 % of upstream llama2.c throughput Xeon Silver 4114, 5 threads

≈ 300 tokens / s MacBook Air M4, 8 threads same bits at every thread count

Hallucinations: fixed. For real.
Pinned, not cured: the same wrong answer, bit for bit, everywhere.
Debuggable, auditable.